

Cache Memory

Week 9 — The Memory Hierarchy, Mapping Functions, and Replacement

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering

Government College of Engineering & Technology (GCET), Ganderbal

August 18, 2026

Where We Left Off (Week 8)

Last week: the CPU talking to slow devices

- ▶ Polling, interrupts, DMA and I/O processors were four answers to one question: a device is far slower than the CPU — *who does the waiting, and what does the waiting cost?*
- ▶ Every answer moved the cost somewhere else; none made it vanish.

The gap Week 8 left open

Once the data is in main memory, the CPU still waits. DRAM is roughly *a hundred times* slower than the processor that reads it. The same speed-mismatch problem, now *inside* the machine.

Roadmap: Six Questions, One Running System

1. **Memory hierarchy** — why fast, big and cheap cannot be bought together, and how locality lets us fake it.
2. **Main memory** — SRAM, DRAM, and the ROM family.
3. **Cache principles** — blocks, lines, tags, hits and misses.
4. **Mapping functions** — direct, fully associative, and k -way set associative, derived field-by-field from the address.
5. **Replacement algorithms** — LRU, FIFO, LFU, Random, Optimal, traced on one common reference string.
6. **Performance** — hit ratio, AMAT, and multi-level effective access time.

Running thread for today

One system — 64 KB main memory, 2 KB cache, 32-byte blocks — and one address, 0x3F8C, carried through all three mapping schemes. Then one 13-reference string carried through all five replacement policies.

First: why can't we just build one big, fast, cheap memory?

Socratic Check: The Memory Wall

Question

A 2 GHz CPU issues one instruction every 0.5 ns. A DRAM access takes about 60 ns. If every instruction needed one memory access, what fraction of the CPU's time would be spent waiting for memory?

- ▶ $60 \div 0.5 = 120$ instruction slots lost per access — the CPU would run at roughly $1/120$ of its rated speed.
- ▶ We cannot make DRAM $120\times$ faster at DRAM prices. So the question changes: can we make the *common case* fast instead of making *all* of memory fast?
- ▶ That is exactly what the memory hierarchy does.

Three Characteristics That Refuse to Cooperate

The design constraint

Across every memory technology, three characteristics move together in a way we cannot escape [2, Ch. 4, printed pp. 112–137]:

- ▶ Faster access time $\Rightarrow$ greater cost per bit.
 - ▶ Greater capacity $\Rightarrow$ smaller cost per bit.
 - ▶ Greater capacity $\Rightarrow$ slower access time.
-
- ▶ **Access time** (latency): time from “read this address” to “here is the data.” **Memory cycle time**: access time plus any recovery the technology needs before the next access.
 - ▶ **Access method**: sequential (tape), direct (disk), random (RAM — every location costs the same), associative (cache tags — compare by *content*, not address) [2, Table 4.1, PDF p. 156].
 - ▶ The designer cannot beat the trade-off. The designer can only arrange technologies so the fast one is used *most of the time*.

Locality of Reference: Why the Trick Works

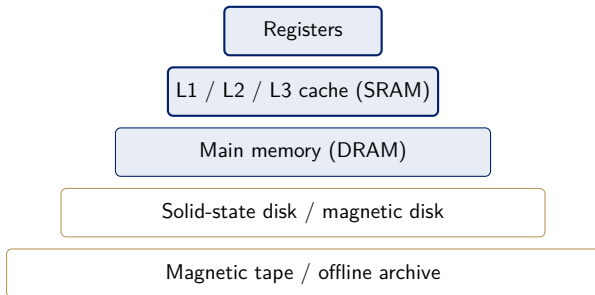
Principle of locality

Memory references made by a running program are not spread uniformly. They cluster [2, §4.1, printed pp. 112–117]:

- ▶ **Temporal locality** — a location referenced now is likely to be referenced again soon (loop counters, the top of the stack).
 - ▶ **Spatial locality** — locations *near* one just referenced are likely to be referenced soon (array traversal, sequential instruction fetch).
-
- ▶ Spatial locality is why we move a whole **block** of neighbouring words on a miss, not just the one word asked for.
 - ▶ Temporal locality is why keeping that block around *pays off* — and why a replacement policy that throws away recently-used data is a bad one (Act 5).

The Hierarchy: Fast at the Top, Big at the Bottom

↑ faster access, smaller capacity, higher cost per bit



↓ slower access, larger capacity, lower cost per bit

What this diagram shows: each level is a cache for the level below it. Registers hold what the current instruction needs; cache holds what the current loop needs; main memory holds what the current program needs [2, Fig. 4.6, PDF p. 160]. Today we study exactly one boundary — the one between cache and main memory.

The Two-Level Performance Model

Average access time across two levels

Let level 1 (cache) have access time T_1 and **hit ratio** H — the fraction of references found in level 1. Let level 2 (main memory) have access time T_2 . Since a miss must *first* search level 1 and only then go to level 2:

$$T_{\text{avg}} = H T_1 + (1 - H) (T_1 + T_2) = T_1 + (1 - H) T_2.$$

- ▶ The whole hierarchy stands or falls on H being close to 1 — and locality is what makes that possible [2, §4.3, Fig. 4.8, PDF p. 162].
- ▶ H is *not* a design knob you set. It is an outcome of cache size, block size, mapping and replacement policy — the rest of this lecture.

Worked Example: What a Hit Ratio Buys

The setup

Cache access $T_1 = 1$ ns, main memory access $T_2 = 100$ ns. Compute T_{avg} at $H = 0.5$, $H = 0.90$ and $H = 0.95$.

- ▶ $H = 0.50$: $T_{\text{avg}} = 1 + 0.50 \times 100 = 51$ ns — barely better than no cache at all.
- ▶ $H = 0.90$: $T_{\text{avg}} = 1 + 0.10 \times 100 = 11$ ns.
- ▶ $H = 0.95$: $T_{\text{avg}} = 1 + 0.05 \times 100 = 6$ ns — a 16.8 $\times$ speed-up over raw DRAM.
- ▶ Notice the shape: going from $H = 0.5$ to $H = 0.9$ saves 40 ns; going from $H = 0.9$ to $H = 0.95$ saves only 5 ns more. The payoff is violently non-linear, which is why real designs fight for the last percent of hit ratio.

Before the cache: what is main memory actually made of?

Two Kinds of Read/Write Memory

SRAM and DRAM — both volatile, both “random access”

- ▶ **SRAM** (static RAM) stores each bit in a flip-flop built from four to six transistors. No refresh needed, fast, expensive, low density. This is what *cache* is built from.
 - ▶ **DRAM** (dynamic RAM) stores each bit as charge on a tiny capacitor — one transistor per cell. Dense and cheap, but the charge leaks, so every cell must be periodically **refreshed**. This is what *main memory* is built from [2, §6.1, PDF pp. 226–236].
-
- ▶ “Random access” means every address costs the same time — it does *not* mean the contents are random.
 - ▶ The SRAM/DRAM split is the hierarchy in miniature: the same speed-versus-density trade-off, resolved twice in the same machine.

The ROM Family: Non-Volatile “Read-Mostly” Memory

Type	How it is written	How it is erased	Volatile?
ROM (mask)	At fabrication, by masking	Not possible	No
PROM	Electrically, once, by the user	Not possible	No
EPROM	Electrically	UV light, whole chip	No
EEPROM	Electrically, byte at a time	Electrically, byte-level	No
Flash	Electrically	Electrically, block-level	No

- ▶ The family is a straight line from **cheapest but unchangeable** (mask ROM, only economic in high volume) to **most flexible but costliest per bit** (EEPROM) [2, §6.1, PDF pp. 226–236].
- ▶ **Flash** sits deliberately in between: erasable in blocks, not bytes — cheaper and denser than EEPROM, more flexible than EPROM.
- ▶ Typical use: boot firmware/BIOS, microcontroller program store, lookup tables, and (as flash) SSDs.

Now the main event: the cache between the CPU and DRAM.

Definition: Cache, Block, Line, Tag

Cache memory

A **cache** is a small, fast SRAM memory placed between the CPU and main memory. Main memory is divided into fixed-size **blocks** of 2^b bytes; the cache holds a small number of **lines**, each able to store *one* block plus a **tag** identifying *which* block it currently holds [2, §5.1, printed pp. 139–142].

- ▶ Because there are far more blocks than lines, the tag is unavoidable: a line cannot be self-identifying.
- ▶ Each line also carries a **valid bit** (has this line ever been loaded?) and, under write-back, a **dirty bit** (has it been modified since loading?).
- ▶ **Hit**: the requested block is present. **Miss**: it is not, so the block is fetched from main memory and installed in a line.

How the CPU Reads Through a Cache

1. The CPU issues an address.
2. The cache splits that address into fields (Act 4) and checks whether the requested block is resident, by comparing tags.
3. **Hit**: the required word is selected from the line using the *word/offset* field and delivered to the CPU. One cache access, done.
4. **Miss**: the cache requests the whole block from main memory, installs it in a line (possibly evicting an existing block — Act 5), and then delivers the word [2, Fig. 5.3, PDF p. 182].

The one thing to hold on to

A miss costs the *whole block transfer*, not one word. That is a bet on spatial locality: we pay for 2^b bytes hoping the neighbours get used too.

Elements of Cache Design

The seven knobs a cache designer actually turns

- ▶ **Cache addresses** — logical (virtual) or physical.
- ▶ **Cache size** — small enough to be fast, big enough to hold the working set.
- ▶ **Mapping function** — direct, associative, set associative. **(Act 4)**
- ▶ **Replacement algorithm** — LRU, FIFO, LFU, Random. **(Act 5)**
- ▶ **Write policy** — write through, write back.
- ▶ **Line size** — how many bytes per block.
- ▶ **Number of caches** — single vs. multi-level, unified vs. split.

This taxonomy is the spine of the whole chapter [2, Table 5.1, PDF p. 184]. The two bolded rows are the two the rest of this lecture is about.

Act 4: given an address, which line may hold it — and how do we know?

The One Question All Three Schemes Answer

Motivation before machinery

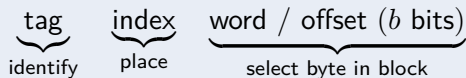
There are far more main-memory blocks than cache lines. So when block number B arrives, we must answer two questions in hardware, fast:

1. **Placement:** which line(s) is B *allowed* to occupy?
 2. **Identification:** given a candidate line, is the block sitting in it actually B ?
- ▶ Restrict placement to *one* line: identification is one tag comparison — cheap, but blocks collide. That is **direct mapping**.
 - ▶ Allow *any* line: no collisions ever, but identification must compare every tag at once — expensive. That is **fully associative mapping**.
 - ▶ Allow *any of k* : the compromise. That is **k -way set associative mapping**.

The Master Framework: Splitting an Address

Derive the fields; never memorise them

Let main memory hold 2^N addressable bytes (so the physical address is N bits), let the block size be 2^b bytes, and let the cache hold $L = \text{cache size} / \text{block size}$ lines.



- ▶ **Direct:** $\text{index} = \log_2 L$ line bits; $\text{tag} = N - \log_2 L - b$.
- ▶ **Fully associative:** no index at all; $\text{tag} = N - b$.
- ▶ **k -way set associative:** $S = L/k$ sets, $\text{index} = \log_2 S$ set bits; $\text{tag} = N - \log_2 S - b$.

The three schemes are one formula with the index field shrinking from $\log_2 L$ bits, through $\log_2(L/k)$, to zero — and every bit the index loses, the tag gains. [2, §5.2, PDF pp. 187–203]

The Running System for Act 4

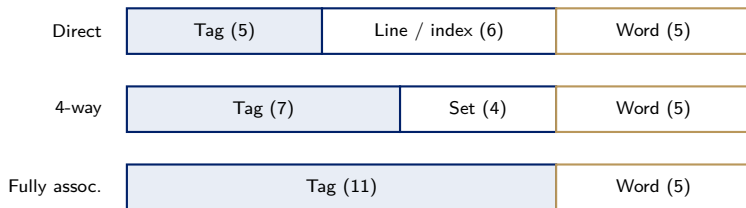
System A — fixed for every worked example that follows

- ▶ Main memory: **64 KB** = 2^{16} bytes $\Rightarrow N = 16$ -bit physical address.
- ▶ Cache: **2 KB** = 2^{11} bytes.
- ▶ Block (line) size: **32 bytes** = $2^5 \Rightarrow b = 5$.

Two derived quantities we will reuse constantly:

- ▶ Number of cache lines $L = 2^{11}/2^5 = 2^6 = \mathbf{64}$ lines.
- ▶ Number of main-memory blocks = $2^{16}/2^5 = 2^{11} = \mathbf{2048}$ blocks.
- ▶ So **2048 blocks compete for 64 lines** — 32 blocks per line. That ratio is the whole reason mapping and replacement exist.
- ▶ Our probe address throughout: $0x3F8C = 16268_{10}$, whose block number is $\lfloor 16268/32 \rfloor = \mathbf{508}$ with byte offset $16268 - 508 \times 32 = \mathbf{12}$.

Three Schemes, One 16-Bit Address



What this diagram shows: the word field never moves — it is fixed by the block size alone. Everything to its left is a fixed budget of 11 bits that the index and tag *share*. Less freedom in placement means a narrower tag; more freedom means a wider one.

Direct Mapping: The Rule

Direct mapping

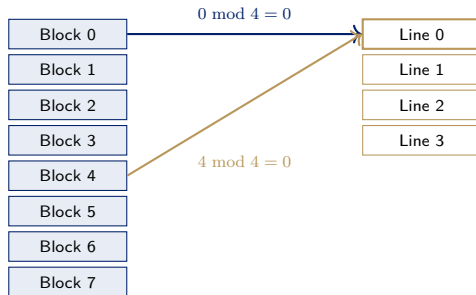
Block B of main memory may occupy *exactly one* cache line:

$$\text{line} = B \bmod L, \quad \text{tag} = \lfloor B/L \rfloor.$$

The line number is read straight out of the address — no search, no choice [2, Fig. 5.7, PDF p. 190].

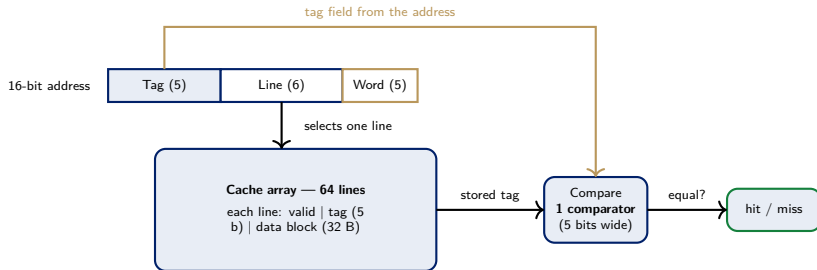
- ▶ Blocks B , $B + L$, $B + 2L$, ... all land on the same line. They are told apart *only* by the tag.
- ▶ **Cheapest possible hardware**: one line decoder, one tag comparator.
- ▶ **The cost**: two hot blocks that happen to be L apart evict each other forever, no matter how empty the rest of the cache is.

How Blocks Collide on One Line



What this diagram shows (toy cache, $L = 4$): blocks 0, 4, 8, 12, ... are all forced onto line 0. Only one may be resident at a time, and the tag records which. Blocks 1, 5, 9, ... queue up for line 1, and so on.

Direct-Mapped Hardware: One Comparator



What this diagram shows: the line field is a *decoder input*, not something to search for. Only after the line is selected does a single comparison decide hit or miss.

Direct Mapping, Worked: Deriving the Bit Widths

System A: 64 KB memory, 2 KB cache, 32-byte blocks

Derive the width of every address field, step by step.

1. Physical address width: $64 \text{ KB} = 2^{16} \text{ bytes} \Rightarrow N = 16$ bits.
2. Word (offset) field: $\text{block} = 32 = 2^5 \text{ bytes} \Rightarrow b = 5$ bits.
3. Number of lines: $L = 2 \text{ KB} / 32 \text{ B} = 2^{11} / 2^5 = 2^6 = 64 \Rightarrow \text{line field} = \log_2 64 = 6$ bits.
4. Tag field $= N - \log_2 L - b = 16 - 6 - 5 = 5$ bits.

Check: $5 + 6 + 5 = 16$ ✓ — the three fields must always re-sum to the full address width. If they do not, one of the four steps above is wrong.

Direct Mapping, Worked: Tracing Address 0x3F8C

Which line does 0x3F8C map to, and what is its tag?

0x3F8C = 0011 1111 1000 1100₂. Slice it 5 | 6 | 5 from the left.

Field	Bits	Value
Tag (5 b)	00111	7
Line (6 b)	111100	60
Word (5 b)	01100	12

Now verify by arithmetic, not by bit-slicing:

- ▶ $0x3F8C = 16268$; block $B = \lfloor 16268/32 \rfloor = 508$; offset = $16268 - 508 \times 32 = 12$ ✓
- ▶ Line = $B \bmod L = 508 \bmod 64 = 508 - 448 = 60$ ✓
- ▶ Tag = $\lfloor B/L \rfloor = \lfloor 508/64 \rfloor = 7$ ✓

Two independent routes to the same answer. In an exam, do one and check with the other.

Direct Mapping's Weakness: Conflict Misses

A two-block loop that defeats a 2 KB cache

A loop alternately touches address 0x0780 (block 60) and 0x0F80 (block 124). Both are in System A. What is the hit ratio?

- ▶ $60 \bmod 64 = 60$ and $124 \bmod 64 = 60$ — **both map to line 60**, with tags $\lfloor 60/64 \rfloor = 0$ and $\lfloor 124/64 \rfloor = 1$.
- ▶ Every reference finds the *other* block's tag in line 60, so every reference is a miss: hit ratio = **0%**, while **63 of the 64 lines sit empty**.
- ▶ This is a **conflict (collision) miss** — not a capacity problem, purely a placement-restriction problem.
- ▶ **Fix:** in a 2-way set-associative cache ($S = 64/2 = 32$ sets), $60 \bmod 32 = 28$ and $124 \bmod 32 = 28$ — still the same set, but now the set has **two** ways, so both blocks stay resident and the hit ratio jumps to nearly **100%**.

Fully Associative Mapping: The Rule

Fully associative mapping

A block may be placed in *any* cache line. There is no index field: the address splits into **tag** | **word** only, and the tag is simply the block number [2, Fig. 5.10, PDF p. 195].

$$\text{tag} = B = \lfloor \text{address} / 2^b \rfloor, \quad \text{tag width} = N - b.$$

- ▶ **No conflict misses ever.** A block is evicted only when the cache is genuinely full — so the only misses left are compulsory and capacity misses.
- ▶ **Identification is now a search.** With no index to narrow the candidates, *every* line's tag must be compared, simultaneously.
- ▶ That is what **content-addressable memory (CAM)** does: a memory searched by content rather than by address [2, PDF pp. 193–194].

Fully Associative, Worked: 0x3F8C Again

Same System A — rederive from scratch

Only the block size changes the word field; everything else the tag absorbs.

1. Word field = $b = 5$ bits (unchanged — 32-byte block).
2. There is no index field, so tag = $N - b = 16 - 5 = 11$ bits.
3. Check: $11 + 5 = 16$ ✓
4. Slice $0x3F8C = 0011\ 1111\ 1000\ 1100_2$ as $11 \mid 5$:
 - ▶ Tag = $00111111100_2 = 508$
 - ▶ Word = $01100_2 = 12$
5. Verify: the tag *is* the block number, and we computed $B = 508$ on the direct-mapping slide ✓

Answer to “which line?": *any of the 64* — that is precisely what fully associative means, and precisely why a replacement policy becomes necessary.

The Price of Full Associativity

Count the hardware for System A

- ▶ Comparators: **64**, one per line, each **11 bits** wide, all firing in parallel on every access.
 - ▶ Tag storage: $64 \times 11 = \mathbf{704}$ bits.
 - ▶ Compare with direct mapping: **1** comparator of **5** bits, and $64 \times 5 = \mathbf{320}$ tag bits.
-
- ▶ Full associativity costs **$2.2\times$ the tag storage** and **$64\times$ the comparators** — and the comparator array is on the critical path, so it also costs *access time*.
 - ▶ Real caches with hundreds or thousands of lines cannot afford this. Hence the compromise on the next slide.

k -Way Set-Associative Mapping: The Rule

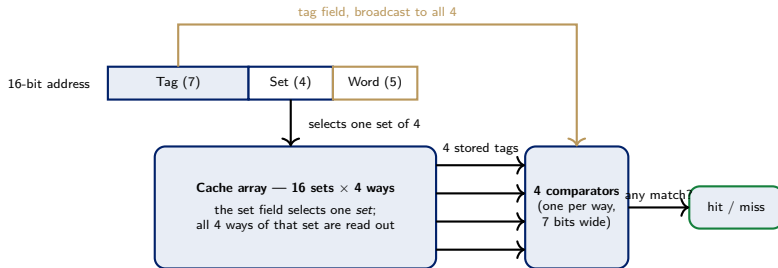
k -way set associative mapping

Group the L lines into $S = L/k$ **sets** of k lines each. A block is mapped to *one* set by index, and may occupy *any* of the k lines (“ways”) inside it [2, Fig. 5.13, PDF p. 200]:

$$\text{set} = B \bmod S, \quad \text{tag} = \lfloor B/S \rfloor, \quad \text{tag width} = N - \log_2 S - b.$$

- ▶ $k = 1$ gives $S = L$: that is exactly **direct mapping**.
- ▶ $k = L$ gives $S = 1$: that is exactly **fully associative mapping**.
- ▶ So set associativity is not a third idea — it is the *dial* between the other two, and $k = 2, 4, 8$ are the settings real designs use.

Set-Associative Hardware: k Comparators



Compare with the direct-mapped figure: identical skeleton, one comparator becomes k , and two bits move from the index field into the tag.

4-Way Set Associative, Worked: Bit Widths

System A again, now organised 4-way

64 KB memory, 2 KB cache, 32-byte blocks, $k = 4$.

1. Address width $N = 16$; word field $b = 5$ (both unchanged — neither depends on associativity).
2. Lines $L = 64$ (unchanged — cache size and block size alone fix it).
3. Number of sets $S = L/k = 64/4 = 16 \Rightarrow$ set field $= \log_2 16 = 4$ bits.
4. Tag $= N - \log_2 S - b = 16 - 4 - 5 = 7$ bits.
5. Check: $7 + 4 + 5 = 16$ ✓

Notice what moved: going from direct ($k=1$) to 4-way, the index lost 2 bits ($6 \rightarrow 4$) and the tag gained exactly those 2 ($5 \rightarrow 7$). Doubling k always trades one index bit for one tag bit.

4-Way Set Associative, Worked: Tracing 0x3F8C

Which set does 0x3F8C map to, and what is its tag?

0x3F8C = 0011 1111 1000 1100₂, sliced 7 | 4 | 5.

Field	Bits	Value
Tag (7 b)	00111111	31
Set (4 b)	1100	12
Word (5 b)	01100	12

Arithmetic cross-check (block $B = 508$, as before):

- ▶ Set = $B \bmod S = 508 \bmod 16 = 508 - 496 = 12$ ✓
- ▶ Tag = $\lfloor B/S \rfloor = \lfloor 508/16 \rfloor = 31$ ✓
- ▶ Word offset = 12, unchanged in all three schemes ✓

The answer to “which line?” is now: *any of the 4 ways of set 12* — which is exactly where a replacement policy is finally needed.

Hardware Cost, Counted

Organisation	Sets S	Index bits	Tag bits	Comparators	Total tag storage
Direct ($k=1$)	64	6	5	1	$64 \times 5 = 320$ b
2-way	32	5	6	2	$64 \times 6 = 384$ b
4-way	16	4	7	4	$64 \times 7 = 448$ b
8-way	8	3	8	8	$64 \times 8 = 512$ b
Fully associative	1	0	11	64	$64 \times 11 = 704$ b

- ▶ All rows are System A — same 64 lines, same 32-byte blocks. Only the *grouping* changes.
- ▶ Comparator count **doubles** with every doubling of k ; tag storage grows only **one bit per line**. The comparators, not the tag bits, are what make full associativity unaffordable.
- ▶ Add one valid bit per line throughout (and one dirty bit under write-back) — constant across all rows, so it does not change the comparison.

The Three Mapping Schemes, Head to Head

	Direct	k -way set associative	Fully associative
Address split	tag line word	tag set word	tag word
Index width	$\log_2 L$	$\log_2(L/k)$	0
Tag width	$N - \log_2 L - b$	$N - \log_2(L/k) - b$	$N - b$
Placement rule	$\text{line} = B \bmod L$	$\text{set} = B \bmod (L/k)$	anywhere
Candidate lines	1	k	all L
Comparators	1	k	L
Replacement policy	none needed	needed, within the set	needed, over the whole cache
Conflict misses	severe	reduced, roughly as k grows	none
Search hardware	decoder only	decoder + k comparators	full CAM
Access time	fastest	intermediate	slowest
Typical use	simple / large L2–L3	L1 and L2 in practice	TLBs, small victim caches

One dial, two end-stops. $k = 1$ and $k = L$ are the extremes of the same formula; every real design lives somewhere in between [2, §5.2, PDF pp. 187–203].

Socratic Check: Choosing an Organisation

Question

An L1 data cache must answer in one CPU cycle. A last-level L3 cache may take twenty cycles but must not waste its (large) capacity. Which associativity would you pick for each, and why?

- ▶ **L1**: low associativity (2- or 4-way). The comparator array is on the critical path, and a 1-cycle budget cannot afford 64 parallel comparisons. Some conflict misses are the accepted price.
- ▶ **L3**: higher associativity (8- or 16-way). Latency is already large, so a wider comparator array is affordable — and with a big cache, wasting capacity to conflicts is the more expensive mistake.
- ▶ The general rule: associativity buys hit ratio and costs access time. Spend it where access time is not the binding constraint.

Act 5: the set is full and a new block arrives. Which resident block dies?

When Is There Anything to Decide?

Replacement exists only where placement offers a choice

- ▶ **Direct mapping needs no replacement algorithm at all.** A block has exactly one legal line; if that line is occupied, the occupant is evicted. There is no decision to make, so there is no policy to implement [2, §5.2 Replacement Algorithms, PDF p. 203].
- ▶ **Fully associative:** the victim is chosen among *all* L lines.
- ▶ **k -way set associative:** the victim is chosen among the k *lines of the target set* — never across sets.
- ▶ To be useful at cache speed, a policy must be implementable *entirely in hardware* — no software bookkeeping, no sorting. That constraint eliminates many policies that look good on paper.

Five Policies

- ▶ **LRU (Least Recently Used)** — evict the line unreferenced for longest. Exploits temporal locality directly; the most popular policy in practice [2, §5.2, PDF p. 203].
- ▶ **FIFO (First In, First Out)** — evict the line resident longest, regardless of use. Implementable as a round-robin/circular buffer.
- ▶ **LFU (Least Frequently Used)** — evict the line with the smallest reference count.
- ▶ **Random** — evict a line chosen at random. Simulation studies report only slightly inferior performance to the usage-based algorithms [2, §5.2, PDF p. 203].
- ▶ **OPT (Optimal / Belady's MIN)** — evict the line whose *next* reference is furthest in the future.

Sourcing note — read this carefully

The first four are Stallings' four cache replacement algorithms. **OPT is not in that text at all**; it is presented here as *standard general treatment, with no page citation*. OPT is also **unimplementable** — it requires knowing the future. Its role is as a *benchmark*: no real policy can beat it, so it bounds how much a better policy could possibly gain.

The Common Reference String

One experiment, five policies

A **fully associative** cache with **3 lines**, initially empty, receives this stream of *block numbers*:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 (13 references)

- ▶ Three lines makes every eviction a genuine three-way choice, so the policies actually diverge — which is the point.
- ▶ In the traces that follow, each column is one reference. The three rows are the three cache lines (slots), M marks a miss and H a hit.
- ▶ The first three references are **compulsory misses**: an empty cache must miss on first touch, whatever the policy. Every policy scores at least 3 misses here, so only the remaining 10 references separate them.

Trace 1 — FIFO

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Slot 0	7	7	7	2	2	2	2	4	4	4	0	0	0
Slot 1	–	0	0	0	0	3	3	3	2	2	2	2	2
Slot 2	–	–	1	1	1	1	0	0	0	3	3	3	3
Hit/Miss	M	M	M	M	H	M	M	M	M	M	M	H	H

- ▶ Victims are taken in strict round-robin order: slot 0, 1, 2, 0, ... — FIFO never looks at whether a block is being used.
- ▶ Reference 7 is the tell: block 0 was referenced only two steps earlier, yet FIFO evicts it because it happened to arrive early.
- ▶ **Result: 3 hits, 10 misses** — hit ratio $3/13 = 23.1\%$.

Trace 2 — LRU

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Slot 0	7	7	7	2	2	2	2	4	4	4	0	0	0
Slot 1	–	0	0	0	0	0	0	0	0	3	3	3	3
Slot 2	–	–	1	1	1	3	3	3	2	2	2	2	2
Hit/Miss	M	M	M	M	H	M	H	M	M	M	M	H	H

- ▶ At reference 6, the resident blocks are $\{2, 0, 1\}$ with block 1 unreferenced since step 3 — so LRU evicts *block 1*, where FIFO evicted block 0.
- ▶ That single better decision buys the extra hit at reference 7.
- ▶ **Result: 4 hits, 9 misses** — hit ratio $4/13 = 30.8\%$, beating FIFO.

Trace 3 — LFU

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Slot 0	7	7	7	2	2	2	2	4	4	3	3	3	3
Slot 1	—	0	0	0	0	0	0	0	0	0	0	0	0
Slot 2	—	—	1	1	1	3	3	3	2	2	2	2	2
Count of 0	—	1	1	1	2	2	3	3	3	3	4	4	4
Hit/Miss	M	M	M	M	H	M	H	M	M	M	H	H	H

- ▶ Ties in count are broken by age (oldest first). Block 0's count climbs to 3 by reference 7, so LFU **protects it** at reference 8 — where LRU evicted it at reference 10.
- ▶ **Result: 5 hits, 8 misses** — hit ratio $5/13 = 38.5\%$, the best of the four implementable policies on *this* string.
- ▶ Do not over-generalise: LFU's weakness is stale popularity — a block referenced heavily long ago keeps a high count and refuses to leave. On a different string LRU wins.

Trace 4 — Random (One Particular Run)

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Slot 0	7	7	7	7	0	3	3	4	4	4	0	0	0
Slot 1	—	0	0	2	2	2	2	2	2	2	2	2	2
Slot 2	—	—	1	1	1	1	0	0	0	3	3	3	3
Hit/Miss	M	M	M	M	M	M	M	M	H	M	M	H	H

- ▶ **Result of this run: 3 hits, 10 misses** — hit ratio 23.1%.
- ▶ **This number is not reproducible.** A different sequence of random victims gives a different count; only the *expected* hit ratio over many runs is a property of the policy.
- ▶ So why use it? It needs **zero bookkeeping hardware** — no counters, no recency list, no age bits — and simulation studies find it only slightly inferior to the usage-based algorithms [2, §5.2, PDF p. 203].

Trace 5 — OPT (the Unattainable Benchmark)

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Slot 0	7	7	7	2	2	2	2	2	2	2	2	2	2
Slot 1	–	0	0	0	0	0	0	4	4	4	0	0	0
Slot 2	–	–	1	1	1	3	3	3	3	3	3	3	3
Hit/Miss	M	M	M	M	H	M	H	M	H	H	M	H	H

- ▶ How the decisions are made: at reference 8 the residents are $\{2, 0, 3\}$; their next uses are block 2 at step 9, block 3 at step 10, block 0 at step 11. **Block 0 is used furthest away, so block 0 is evicted** — the opposite of what LFU chose.
- ▶ **Result: 6 hits, 7 misses** — hit ratio $6/13 = 46.2\%$.
- ▶ Standard general treatment — not from Stallings, which does not cover OPT. Use it as a yardstick, never as a design.

The Five Policies, Same Input, Different Outcomes

Policy	Hits	Misses	Hit ratio	What it tracks
OPT (benchmark)	6	7	46.2%	the future (unimplementable)
LFU	5	8	38.5%	a reference counter per line
LRU	4	9	30.8%	recency order of the lines
FIFO	3	10	23.1%	load order only
Random (this run)	3	10	23.1%	nothing

- ▶ Identical input, identical cache, **a 2× spread in hit ratio** — the policy is not a detail.
- ▶ OPT's 46.2% is the ceiling: no implementable policy on this string can do better, so LFU's 38.5% leaves at most 7.7 percentage points on the table.
- ▶ One string proves nothing about policies in general. It proves that policies diverge — which is why real designs are chosen by simulation over real traces, not by argument.

Belady's Anomaly: More Cache, More Misses

The claim that ought to be impossible

Under **FIFO**, *increasing* the number of cache lines can *increase* the number of misses on the same reference string. This is **Belady's anomaly**.

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 — FIFO with 3 lines:

Reference	1	2	3	4	1	2	5	1	2	3	4	5
Slot 0	1	1	1	4	4	4	5	5	5	5	5	5
Slot 1	–	2	2	2	1	1	1	1	1	3	3	3
Slot 2	–	–	3	3	3	2	2	2	2	2	4	4
Hit/Miss	M	M	M	M	M	M	M	H	H	M	M	H

3 lines $\Rightarrow$ 9 misses, 3 hits. Now give the cache one more line and count again on the next slide. Standard general treatment: Belady's anomaly appears nowhere in this course's Stallings text, so no page is cited for it.

Belady's Anomaly: The Punchline

Same string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 — FIFO with 4 lines:

Reference	1	2	3	4	1	2	5	1	2	3	4	5
Slot 0	1	1	1	1	1	1	5	5	5	5	4	4
Slot 1	–	2	2	2	2	2	2	1	1	1	1	5
Slot 2	–	–	3	3	3	3	3	3	2	2	2	2
Slot 3	–	–	–	4	4	4	4	4	4	3	3	3
Hit/Miss	M	M	M	M	H	H	M	M	M	M	M	M

The result

3 lines: 9 misses. 4 lines: 10 misses. A *bigger* cache performed *worse* on the identical reference string.

- ▶ **LRU and OPT are immune.** They are *stack algorithms*: the set of blocks held with n lines is always a subset of the set held with $n + 1$ lines, so extra capacity can never remove a block that would otherwise have hit.
- ▶ FIFO has no such property — adding a line changes the eviction order itself, not just its depth.

Implementing These Policies in Hardware

Each policy is only as good as the hardware it needs

- ▶ **LRU, two-way set associative** — one **USE bit** per set. Referencing way 0 sets the bit to 0, referencing way 1 sets it to 1; the victim is the way the bit does *not* point at. One bit per set, total [2, §5.2, PDF p. 203].
- ▶ **LRU, fully associative** — maintain an **index list** of all L lines in recency order. A reference moves that line to the front; the victim is the tail. Correct, but the list must be reordered on *every* access, which is why exact LRU is rare beyond small associativities [2, §5.2, PDF p. 203].
- ▶ **FIFO** — a **circular buffer** (round-robin pointer) per set. Advance the pointer on each replacement; no update at all on a hit. $\lceil \log_2 k \rceil$ bits per set.
- ▶ **LFU** — a **reference counter** per line, incremented on every access to that line; the victim is the minimum. Costs a counter and a comparison tree per set, and needs periodic ageing to stop stale counts dominating.
- ▶ **Random** — a free-running counter or LFSR shared by the whole cache. **No per-line state whatsoever**, which is exactly its appeal.

The pattern: cost rises with how much history the policy remembers. Random remembers nothing; exact LRU remembers everything.

Socratic Check: Why Is 2-Way LRU So Cheap?

Question

Exact LRU over 64 fully associative lines needs a 64-entry ordered list. Exact LRU over a 2-way set needs *one bit*. Where did the cost go?

- ▶ Recency order over n items is a *permutation*: there are $n!$ of them, needing $\lceil \log_2 n! \rceil$ bits. For $n = 2$ that is $\lceil \log_2 2 \rceil = 1$ bit; for $n = 64$ it is roughly **296 bits per set**.
- ▶ So associativity does not merely cost comparators (Act 4) — it costs **replacement bookkeeping too**, and that cost grows far faster than linearly.
- ▶ This is why real 8- and 16-way caches use *pseudo*-LRU (a small tree of bits that approximates recency) rather than exact LRU.

Act 6: put a number on all of it — hit ratio, AMAT, and two cache levels.

Hit Ratio, Miss Ratio, and AMAT

The three definitions everything else is built from

For n memory references of which n_h hit in the cache:

$$H = \frac{n_h}{n}, \quad \text{miss ratio} = 1 - H,$$

$$\text{AMAT} = T_{\text{hit}} + (1 - H) \times T_{\text{miss penalty}}.$$

AMAT is the **average memory access time** seen by the CPU [1, §5.3, p. 397].

- ▶ **Reconciling the two textbook forms.** Act 1 wrote $T_{\text{avg}} = HT_1 + (1 - H)(T_1 + T_2)$. Expanding gives $T_1 + (1 - H)T_2$ — identical to AMAT with $T_{\text{hit}} = T_1$ and miss penalty $= T_2$.
- ▶ The trap: a miss penalty is the **extra** time *beyond* the hit time, because the hit time is paid on every access, hit or miss. Adding it twice is the single most common arithmetic error here.

Worked Example: AMAT from a Reference Count

The setup

A program makes 1000 memory references; 940 are found in the cache. Cache access time is 2 ns and a miss costs a further 80 ns to fetch the block from main memory. Find the hit ratio, miss ratio and AMAT.

- ▶ Hit ratio $H = 940/1000 = 0.94$; miss ratio $= 1 - 0.94 = 0.06$.
- ▶ $AMAT = 2 + 0.06 \times 80 = 2 + 4.8 = 6.8$ ns.
- ▶ Cross-check with the Act 1 form:
 $0.94 \times 2 + 0.06 \times (2 + 80) = 1.88 + 4.92 = 6.8$ ns ✓
- ▶ Sanity test on any AMAT answer: it must lie strictly between the hit time (2 ns) and the full miss time (82 ns). 6.8 does.

Two Cache Levels: Two Access Models

The distinction that decides the arithmetic

With an L1 cache (time t_1 , hit ratio h_1), an L2 cache (time t_2 , *local* hit ratio h_2 measured over L1 misses only), and main memory (time t_m):

- **Hierarchical (sequential) access** — each level is searched only after the one above it misses, so its time *adds*:

$$T = h_1 t_1 + (1 - h_1) h_2 (t_1 + t_2) + (1 - h_1)(1 - h_2)(t_1 + t_2 + t_m).$$

- **Simultaneous access** — levels are probed in parallel, so only the level that answers contributes:

$$T = h_1 t_1 + (1 - h_1) h_2 t_2 + (1 - h_1)(1 - h_2) t_m.$$

Always read the question for which model is intended. Unless a problem says otherwise, assume hierarchical — it is what real multi-level caches do [2, §5.2 Number of Caches, PDF pp. 205–208].

Worked Example: Effective Access Time, Hierarchical

The setup

$t_1 = 1$ ns with $h_1 = 0.95$; $t_2 = 10$ ns with local $h_2 = 0.90$; main memory $t_m = 100$ ns.
Find the effective access time.

- ▶ L1 hit: $0.95 \times 1 = 0.95$ ns.
- ▶ L1 miss, L2 hit: $(1 - 0.95) \times 0.90 \times (1 + 10) = 0.045 \times 11 = 0.495$ ns.
- ▶ Both miss: $(1 - 0.95) \times (1 - 0.90) \times (1 + 10 + 100) = 0.005 \times 111 = 0.555$ ns.
- ▶ **Total $T = 0.95 + 0.495 + 0.555 = 2.0$ ns.**
- ▶ Weight check: $0.95 + 0.045 + 0.005 = 1.000$ ✓ — the three probabilities must sum to 1, or a case has been missed.
- ▶ Compare with 100 ns raw DRAM: the two-level hierarchy delivers a **50×** effective speed-up.

Same Numbers, Simultaneous Model — and a Warning

Recompute with parallel probing

$$T = 0.95(1) + 0.045(10) + 0.005(100) = 0.95 + 0.45 + 0.50 = \mathbf{1.9} \text{ ns.}$$

- ▶ Only 0.1 ns apart here — but the gap widens fast as h_1 falls, because every miss then pays t_1 twice as often under the hierarchical model.
- ▶ **The other classic trap: local vs. global hit ratio.** Here $h_2 = 0.90$ is *local* — 90% of the references that *reach* L2. The **global** L2 hit ratio is $(1 - h_1)h_2 = 0.05 \times 0.90 = 0.045$, i.e. only 4.5% of all references.
- ▶ If a problem states an L2 hit ratio without saying which, the phrase “of the accesses that miss in L1” marks it local; “of all memory references” marks it global. Using the wrong one is a full-marks error, not a rounding one.

Socratic Check: Where Should the Money Go?

Question

Starting from $t_1 = 1$ ns, $h_1 = 0.95$, $t_m = 100$ ns and no L2 (so $\text{AMAT} = 1 + 0.05 \times 100 = 6$ ns), you may spend your budget on exactly one of: (a) halving t_1 to 0.5 ns, or (b) raising h_1 to 0.975. Which wins?

- ▶ (a) $0.5 + 0.05 \times 100 = 5.5$ ns — a 0.5 ns saving.
- ▶ (b) $1 + 0.025 \times 100 = 3.5$ ns — a 2.5 ns saving, **five times better**.
- ▶ **The lesson:** when the miss penalty is large, the miss *rate* dominates AMAT, and hit time is a second-order term. Halving the miss rate is worth more than halving the hit time.
- ▶ This is exactly why Act 4 spent so long on mapping and Act 5 on replacement: both are miss-rate levers.

Two Design Knobs We Have Not Yet Priced

Write policy — what happens when the CPU writes?

- ▶ **Write through**: every write updates cache *and* main memory. Memory is always consistent; write traffic is high.
 - ▶ **Write back**: writes update only the cache and set the line's **dirty bit**; main memory is updated only when a dirty line is evicted. Far less traffic, but memory is temporarily stale — a real problem for I/O modules and for other processors' caches [2, §5.2 Write Policy, PDF pp. 203–205].
-
- ▶ **Line size** has a maximum, not a monotone benefit: larger blocks exploit spatial locality, but past some point they fetch data that is never used and evict data that still is, so the hit ratio turns back down [2, §5.2 Line Size, PDF p. 205].
 - ▶ Both knobs change the miss *penalty* and the miss *rate* — the same two terms Act 6's AMAT formula is built from.

Six questions, one idea. What is the pattern underneath?

The Threads Meet

Every cache decision buys hit ratio and pays in hardware or time

- ▶ **Hierarchy vs. one memory:** locality lets a 2 KB SRAM stand in for 64 KB of DRAM most of the time (recall: $H = 0.95$ turned 100 ns into 6 ns).
- ▶ **Mapping (Act 4):** direct mapping costs 1 comparator and 320 tag bits but suffers 0% hit ratio on a two-block conflict loop; fully associative removes every conflict miss and costs 64 comparators and 704 tag bits. Set associativity is the dial between them.
- ▶ **Replacement (Act 5):** on one identical 13-reference string, FIFO scored 3 hits, LRU 4, LFU 5, and the unattainable OPT 6 — and FIFO can be made *worse* by a bigger cache (Belady).
- ▶ **Performance (Act 6):** AMAT is dominated by miss rate when the miss penalty is large — which is why Acts 4 and 5 mattered.

Exactly the pattern Weeks 5–8 established for buses, control units and I/O: **no free lunch, only a choice of who pays.**

Summary

- ▶ **Hierarchy**: capacity, cost/bit and access time cannot all improve together; **temporal and spatial locality** make a small fast level useful. Two-level model $T_{\text{avg}} = T_1 + (1 - H)T_2$.
- ▶ **Main memory**: SRAM (flip-flop, no refresh, cache) vs. DRAM (capacitor, refresh, main memory); the ROM family runs from mask ROM (unchangeable) through PROM, EPROM (UV erase), EEPROM (byte erase) to flash (block erase).
- ▶ **Mapping** — one formula, three settings: tag | index | word, with index = $\log_2 L$ (direct), $\log_2(L/k)$ (k -way), or 0 bits (fully associative), and the tag absorbing every bit the index gives up. For System A: 5 | 6 | 5, 7 | 4 | 5, 11 | 5; address 0x3F8C → line 60/tag 7, set 12/tag 31, tag 508.
- ▶ **Cost**: comparators = 1, k , or L — doubling associativity doubles the comparators but adds only one tag bit per line.
- ▶ **Replacement**: needed only where placement offers a choice (*never* under direct mapping). LRU (USE bit / index list), FIFO (circular buffer), LFU (per-line counter), Random (no state), and OPT as a benchmark. **Belady's anomaly**: FIFO can miss *more* with a larger cache; LRU and OPT, being stack algorithms, cannot.
- ▶ **Performance**: $\text{AMAT} = T_{\text{hit}} + (1 - H) \times \text{miss penalty}$; multi-level effective access time depends on whether access is hierarchical or simultaneous, and on whether h_2 is local or global.

Bridge to Week 10

The same idea, one level down the hierarchy

Week 9 made a small fast memory stand in for a large slow one, using blocks, tags, mapping and replacement. Week 10 applies the identical machinery between *main memory and disk* — and calls it **virtual memory**.

- ▶ Cache line → page; tag lookup → page table; miss → page fault; and the TLB is itself a small associative cache of translations.
- ▶ Replacement returns too — and this time Belady's anomaly is not a curiosity but a documented hazard of FIFO page replacement.
- ▶ We will also meet segmentation, and secondary-storage organisation including RAID.

References



David A. Patterson and John L. Hennessy.

Computer Organization and Design: The Hardware/Software Interface.

Morgan Kaufmann, arm edition (5th) edition, 2017.



William Stallings.

Computer Organization and Architecture: Designing for Performance.

Pearson, 10th edition, 2015.

Anchor reading for this week: Stallings 11e Ch. 4 (memory hierarchy), Ch. 5 §5.1–5.2 (cache organisation, mapping, replacement), Ch. 6 §6.1 (ROM types); Patterson & Hennessy Ch. 5 §5.3–5.4 (tag/index/offset, AMAT); Hamacher Ch. 8 (chapter-level only — not page-verified for this course). Optimal replacement and Belady's anomaly are standard general treatment and are deliberately given no page citation.